CSS Cheatsheet

PRAVAN · Chapter 3 · Styling static sites

WHY CSS

HTML describes the **content**. CSS (Cascading Style Sheets) describes how it **looks**: colours, fonts, spacing, layout, animations.

- Easier maintenance
- Same look across all pages from one file
- Several styles for one document
- Lighter pages: the stylesheet is loaded once, then cached

ANATOMY OF A RULE

```
h1 {
  color: red;          /* property: value; */
  font-size: 24px;
}
^ selector   ^ declaration block in { }
```

A stylesheet is a list of rules. A rule is a **selector** plus one or more **declarations** (`property: value;`).

WHERE TO PUT CSS

METHOD	HOW
Inline	`<h1 style="color:red">`
In the head	`<style> … </style>` inside `<head>`
External file	`<link rel="stylesheet" href="style.css">` in `<head>`

> External file is the best practice.

Selectors

Choosing which elements to style

THE BIG THREE

TYPE	SYNTAX	TARGETS
Element	`p { }`	Every `<p>`
Class	`.title { }`	Every element with `class="title"` (reusable)
Id	`#intro { }`	The one element with `id="intro"`

> Same style for many elements → class. A single element → id.

HIERARCHY & COMBINATORS

`#box p`	Any `p` inside `#box` (descendant)
`ul > li`	Direct children only
`h1 + p`	The `p` right after an `h1`
`h1 ~ p`	All `p` siblings after an `h1`
`a[href=""]`	By attribute

PSEUDO-CLASSES & PSEUDO-ELEMENTS

`a:hover`	While the mouse is over it
`a:active` `a:visited`	While clicked / already visited
`p::before` `p::after`	Inject content before / after, e.g. `content: "*"`

More selectors: developer.mozilla.org/en-US/docs/Web/CSS/CSS_selectors

WHICH RULE WINS?

1. **Context:** inline → head → external file
2. **Specificity:** id → class → element
3. **Tie:** the last one written wins

Resolved per property. Inline overrides the rest.

Text & colours

The properties you'll reach for first

COLOUR

`color`	Text colour
`background-color`	Background colour
`background-image`	`url(img.jpg)` or `linear-gradient(0deg, white, blue)`

```
color: red;   color: #FF9935;
color: rgb(100,255,33);   color: rgba(100,255,33,0.5);  /* alpha 0 to 1 */
```

TEXT

`font-family`	List from specific to generic: `"Georgia", serif`. Google Fonts also works.
`font-size`	`px` fixed · `em` relative to parent · `rem` relative to root
`font-weight`	`normal` / `bold`
`font-style`	`normal` / `italic`
`text-decoration`	`none` (default) · `underline` · `line-through`
`text-align`	`left` (default) · `center` · `right`
`line-height`	Line spacing, default `1.0`; try `1.5`

> Web-safe fonts: Arial, Verdana, Tahoma, Georgia, Times New Roman, Courier New.

LISTS & TABLES

`list-style-type` changes bullets/numbers · `border` draws borders · `border-collapse: collapse` merges table borders.

Boxes & display

Every element is a box

THE BOX MODEL

```
margin   outside space, between elements
border   the line around the box
padding  inside space, between border and content
content  the text or image itself
```

`padding: 10px`	All four sides
`padding: 10px 20px`	Top & bottom, then left & right
`padding: 5px 10px 15px 20px`	Top, right, bottom, left (clockwise)
`margin: 0 auto`	Centers a block horizontally

`margin` works like `padding`. `width` and `height` size the **content only**, and don't apply to inline elements. Use `max-width` to avoid surprises.

> Open the browser dev tools to see the box model and tweak padding and margin live.

DISPLAY

block
Full width, has width/height.
Default for `h1`, `p`

inline
Content width, no width/height.
Default for `span`

`inline-block` mixes both: side by side, but sizable. Handy for menus.

Layout

Positioning, Flexbox, Grid, and responsive

POSITION

`static`	Default. Normal flow. `top/left…` ignored.
`relative`	Offset from its normal spot
`absolute`	Placed against the nearest `relative` parent
`fixed`	Stays put while scrolling
`sticky`	Switches between relative and fixed on scroll

`z-index` sets stacking order for overlaps. `float` is legacy: prefer Flexbox or Grid.

FLEXBOX & GRID

```
.menu {
  display: flex;
  flex-direction: row;
  justify-content: center;
  gap: 5px;
}
```

Flex: one direction (menus, rows)

```
.wrap {
  display: grid;
  grid-template-columns:
    1fr 1fr 1fr;
  gap: 10px;
}
```

Grid: rows and columns (page layout)

`justify-content`: `start` `end` `center` `space-between`. For Grid: `grid-area`, `fr` units, `vh` / `vw`.

RESPONSIVE

```
@media screen and (max-width: 768px) {
  /* styles for tablets */
}
```

Media queries react to the **viewport** size (not the screen). Common breakpoints: **375px** phone, **768px** tablet, **1024px** desktop.

Effects & habits

Finishing touches and good practice

EFFECTS

`box-shadow`	`2px 4px 8px 3px grey`
`visibility`	`hidden` hides but keeps space
`transition`	`transition: 0.5s ease;` with a `:hover` state
`animation`	`animation: move 2s infinite alternate;` + `@keyframes move { from{…} to{…} }`

GOOD HABITS

Organisation

External `.css` file
Clear, consistent class names
Avoid long, over-specific selectors
General styles first, specifics later

Code

Indent, one property per line
Always end with `;`
Lowercase
Comments: `/* note */`

> Prefer relative units (`em` `rem` `%` `vh` `vw`), Flexbox/Grid over float, and always plan for `@media` .

WHERE CSS IS GOING

- **CSS1** (1996) basics → **CSS2** (1998) layout → **CSS3** (2011) shadows, animation, Flex, Grid, responsive. Now a living standard.
- Modern: variables `:root { --c: red }` + `var(--c)` , nesting, `:has()` , Subgrid, `dvh` units.
- Frameworks save time: Bootstrap, Tailwind, Bulma, Materialize.

HANDY LINKS

w3schools.com/cssref · developer.mozilla.org/en-US/docs/Web/CSS · caniuse.com